॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

CS101C: Introduction to Programming (Using C)

Instructors: Prof. Aikta Arya and Prof. Konjengbam Anand

CSE Department

Indian Institute of Technology, Dharwad

Week 11

Today's Class (21/10/2026)

`fgets()` — Read a Whole Line!

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Imagine a file named `story.txt` containing:

Contents of `story.txt`

```
Once upon a time,
there lived a programmer.
The programmer loved C.
```

Suppose we want to read the **first line** from this file.

One way: Use `getc()`

We could read one character at a time:

$$\texttt{'O'} \rightarrow \texttt{'n'} \rightarrow \texttt{'c'} \rightarrow \cdots$$

But we would need a loop and extra logic to build the line.

`fgets()` — Read a Whole Line!

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Imagine a file named `story.txt` containing:

Contents of `story.txt`

```
Once upon a time,
there lived a programmer.
The programmer loved C.
```

Suppose we want to read the **first line** from this file.

One way: Use `getc()`

We could read one character at a time:

$$\texttt{'O'} \rightarrow \texttt{'n'} \rightarrow \texttt{'c'} \rightarrow \cdots$$

But we would need a loop and extra logic to build the line.

Can we read a whole line in one function call?

Meet `fgets()`!

`fgets()` reads a sequence of characters from a file and stores them in a character array.

Understanding `fgets()`

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Syntax

```
fgets(line, maxlen, fp)
```

`line`	`maxlen`	`fp`
Character array	Array capacity	Input stream

- `line`: the character array where input is stored.
- `maxlen`: the array's capacity, including space for the terminating `'\0'`.
- `fp`: the file stream from which characters are read.

Important

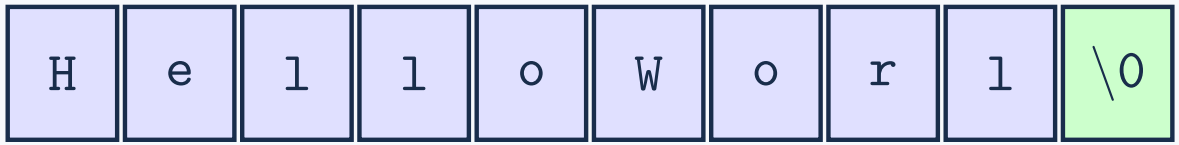
If the array has room for 20 characters, `fgets()` reads at most **19 input characters**, leaving one position for `'\0'`.

Why `maxlen - 1`?

Suppose we declare:

```
char line[10];
```

The array has **10 positions**.



At most 9 input characters String terminator

Why reserve the last position?

C strings end with a special character called the **null terminator**, written as `'\0'`. It tells C where the string ends.

$$\boxed{\text{Maximum input characters} = \texttt{maxlen} - 1}$$

Remember

`fgets()` leaves space for `'\0'` so the stored characters can form a valid C string.

When Does `fgets()` Stop Reading?

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

`fgets()` stops when the first of these happens:

1. It reads a newline character, `'\n'`.
2. It has read at most `maxlen - 1` characters.
3. It encounters the end of the file before reading more.

Example

Suppose `story.txt` contains:

```
Once upon a time,
there lived a programmer.
```

If the array is large enough, the first call to `fgets()` reads:

```
Once upon a time,\n
```

A subtle but important point

If a newline is read, `fgets()` keeps it in the array. It does not automatically remove the newline.

Read the First Line from a File

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

```
#include <stdio.h>
int main(void)
{
    FILE *fp = fopen("story.txt", "r");
    if (fp == NULL)
    {
        printf("Cannot open file.\n");
        return 1;
    }
    char line[100];
    if (fgets(line, sizeof(line), fp) != NULL)
    {
        printf("%s", line);
    }

    fclose(fp);
    return 0;
}
```

Read the First Line from a File

Output, assuming the first line fits

```
Once upon a time,
```

Why check against `NULL`?

On success, `fgets()` returns `line`. If no characters are read because of end-of-file, or a read error occurs, it returns `NULL`.

Why only the first line?

`fgets()` stops at the first newline it reads, unless the buffer limit is reached first or the input ends. It does not automatically read the entire file.

The Newline Mystery

Suppose the file contains:

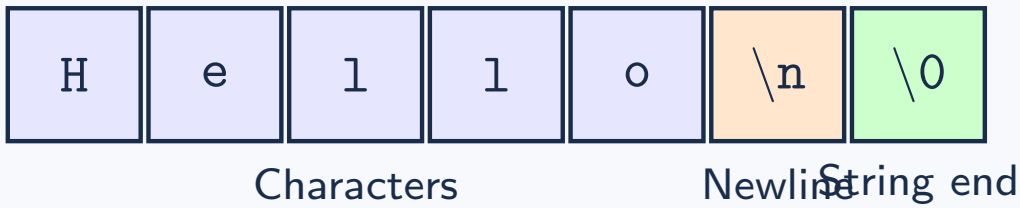
```
Hello
World
```

Two lines in the file

We read the first line:

```
fgets(line, sizeof(line), fp);
```

If the first line fits, the array typically contains:



Important

`fgets()` usually keeps the newline character `'\n'` if it reads one. `'\0'` is added to terminate the C string.

Why Is There an Extra Blank Line?

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Suppose `line` already contains:

`Hello\n\0`

Option A

```
printf("%s", line);
```

```
Hello
```

The stored newline moves output to the next line.

Option B

```
printf("%s\n", line);
```

```
Hello
```

The stored newline plus the additional newline can leave an extra blank line.

Rule of thumb

If you print a string read by `fgets()`, use `printf("%s", line)` when you want to preserve the line ending already stored in the string.

Predict the Output!

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Assume `input.txt` contains:

```
RED
BLUE
GREEN
```

What happens when this code runs?

```
char line[10];
fgets(line, sizeof(line), fp);
printf("%s", line);
fgets(line, sizeof(line), fp);
printf("%s", line);
```

Choose your answer

A. Only RED is printed.

B. RED and BLUE are printed on separate lines.

C. RED BLUE GREEN are all printed.

D. The program cannot compile.

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Correct Answer: B

Output

```
RED
BLUE
```

Trace the file position

1. First `fgets()` reads `RED` and its newline.
2. The file position is now at the start of `BLUE`.
3. Second `fgets()` reads `BLUE` and its newline.
4. Both strings are printed using `printf("%s", line)`.

Why not GREEN?

The program calls `fgets()` only twice. Each call reads at most one line here, so the third line is never read.

Read the Entire File with `fgets()`

Suppose `story.txt` contains:

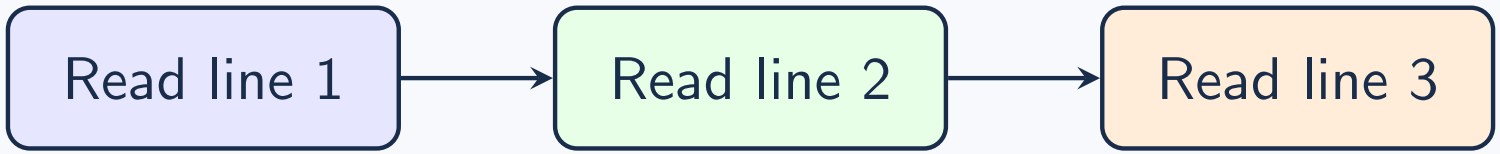
File contents

```
Once upon a time,
there lived a programmer.
The programmer loved C.
```

Important question

One call to `fgets()` reads at most one line (or one chunk if the line is too long).

How do we read the entire file?



Repeat until no more input can be read

The solution

Use a `while` **loop** to call `fgets()` repeatedly until it returns `NULL`.

The Complete Program

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

```
#include <stdio.h>
int main(void)
{
    FILE *fp = fopen("story.txt", "r");
    char line[100];
    if (fp == NULL)
        return 1;
    while (fgets(line, sizeof(line), fp) != NULL)
    {
        printf("%s", line);
    }
    fclose(fp);
    return 0;
}
```

The Complete Program

Understand the loop condition

`fgets(line, sizeof(line), fp) != NULL`

- If reading succeeds, `fgets()` returns `line`.
- The condition is true, so the loop body executes.
- When no more characters can be read, `fgets()` returns `NULL`, and the loop stops.

Follow Each `fgets()` Call

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Suppose the file contains three lines and each line fits in the array.

Call	What is read?	What happens?
1	First line	Printed
2	Second line	Printed
3	Third line	Printed
4	No more input	Returns `NULL`; stop

Output

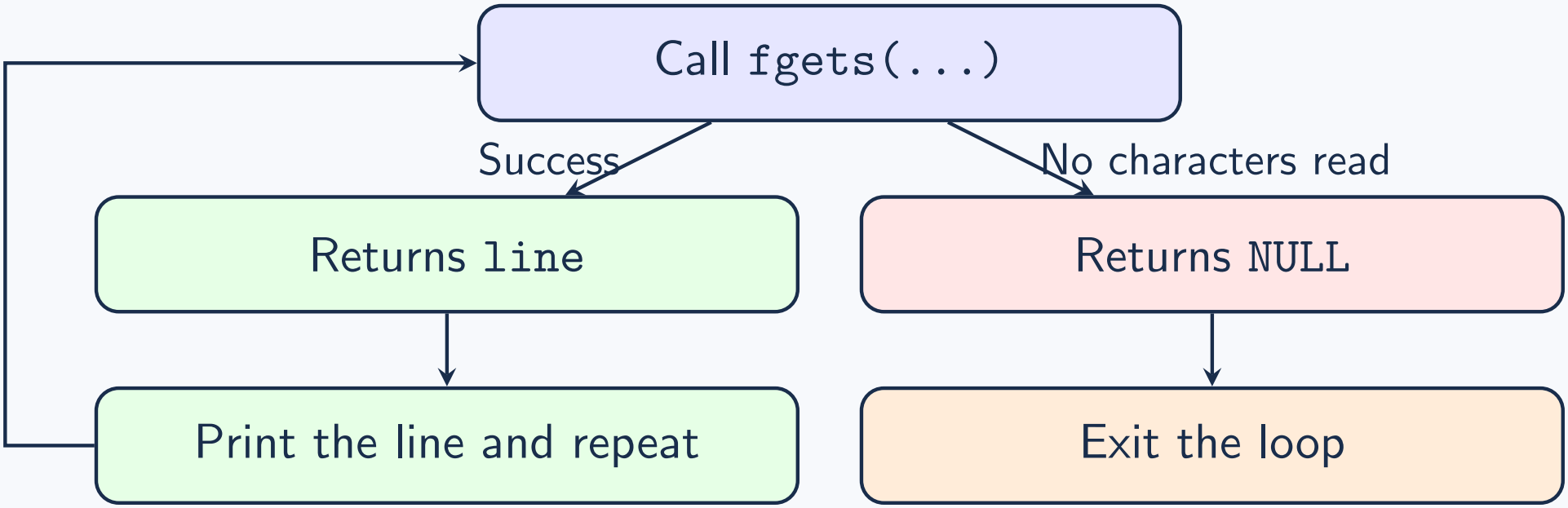
```
Once upon a time,
there lived a programmer.
The programmer loved C.
```

Notice

The program does not need to know how many lines the file contains. The return value of `fgets()` controls the loop.

Why Does the Loop Stop at NULL?

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad



One important detail

`NULL` indicates that `fgets()` did not return a successfully read string. This can happen at end-of-file before any characters are read, or because of a read error. Use `ferror(fp)` if you need to distinguish a read error from end-of-file.

Quick Check!

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Suppose `story.txt` contains five lines.
How many times can the loop body execute?

```
while (fgets(line, sizeof(line), fp) != NULL)
{
    printf("%s", line);
}
```

A. Exactly once
B. Exactly twice
C. Five times, if each line fits in the array
D. It always executes forever

Quick Check!

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Suppose `story.txt` contains five lines.
How many times can the loop body execute?

```
while (fgets(line, sizeof(line), fp) != NULL)
{
    printf("%s", line);
}
```

A. Exactly once
B. Exactly twice
C. Five times, if each line fits in the array
D. It always executes forever

Answer: C

If each line fits in the array, each successful call reads one line. Five lines therefore require five successful calls. A later call returns `NULL`, and the loop stops.

Today's Class (23/10/2026)

fputs() — Write a String to a File

Writing a long sentence one character at a time is tedious!

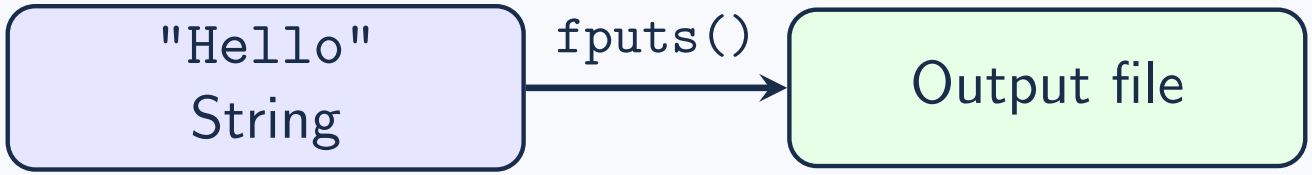
॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Using putc() to write Hello

```
putc('H', fp);
putc('e', fp);
putc('l', fp);
putc('l', fp);
putc('o', fp);
```

Five function calls just to write five characters!

Can we write the entire string in one function call?



Meet fputs()!

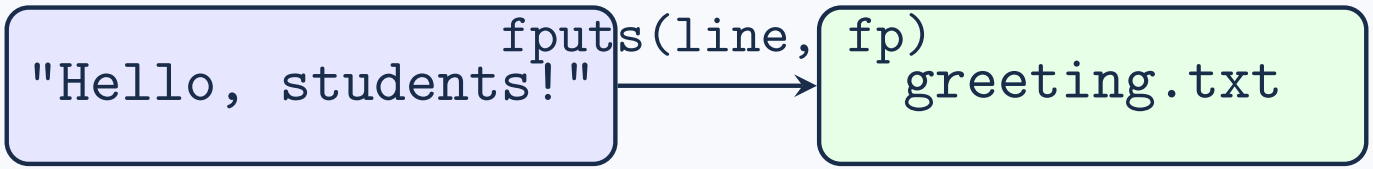
fputs() writes a string to an output stream.

Understanding `fputs()`

Syntax

```
fputs(line, fp)
```

- `line`: the string to write.
- `fp`: the output file stream.



Return value

- Returns a non-negative value on success.
- Returns `EOF` if a write error occurs.

Important

The exact non-negative value returned on success is not specified. Do not assume it equals the string length.

Let's Write Two Sentences!

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

```c
#include <stdio.h>
int main(void)
{
    FILE *fp = fopen("greeting.txt", "w");
    if (fp == NULL)
        return 1;
    fputs("Hello, students!\n", fp);
    fputs("Welcome to C programming.\n", fp);
    fclose(fp);
    return 0;
}
```

Contents of greeting.txt

```
Hello, students!
Welcome to C programming.
```

What does \n do?

It adds a newline character, so the next sentence begins on the following line.

Does `fputs()` Add a Newline?

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Consider these two statements:

Without `\n`

```
fputs("Hello", fp);
fputs("World", fp);
```

```
HelloWorld
```

With `\n`

```
fputs("Hello\n", fp);
fputs("World\n", fp);
```

```
Hello
World
```

Remember

`fputs()` does **not** automatically add a newline.
Add `\n` yourself whenever you need one.

Predict the File Contents!

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

What will `output.txt` contain?

```
FILE *fp = fopen("output.txt", "w");
fputs("Good", fp);
fputs("Morning", fp);
fputs("Everyone\n", fp);
fclose(fp);
```

Choose your answer

A. `Good Morning Everyone`

B. `GoodMorningEveryone`

C. Each word appears on a separate line.

D. Nothing is written.

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Correct Answer: B

Contents of `output.txt`

```
GoodMorningEveryone
```

Trace the three calls

1. `fputs("Good", fp)` writes `Good`.
2. `fputs("Morning", fp)` writes immediately after it.
3. `fputs("Everyone\n", fp)` writes immediately after that, followed by a newline.

Golden rule

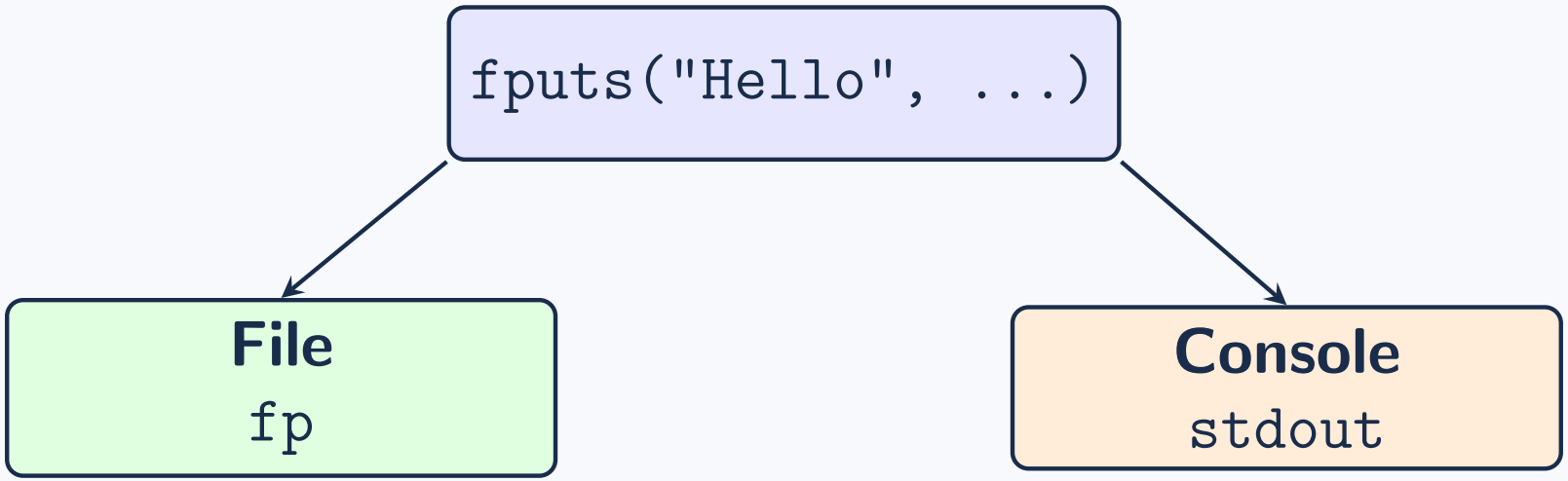
`fputs()` writes exactly the string's characters. It does not automatically insert spaces or newlines.

Where Does `fputs()` Write?

Recall the syntax:

```
fputs(string, stream);
```

The **second argument** tells `fputs()` where to send the string.



Key Idea

Same function, different destination!

Example 1: Writing to a File

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

```c
#include <stdio.h>

int main(void)
{
    FILE *fp = fopen("output.txt", "w");
    if (fp == NULL)
        return 1;
    fputs("Hello students!\n", fp);
    fclose(fp);
    return 0;
}
```

What happens?

- `fopen()` opens `output.txt`.
- `fp` represents the opened file stream.
- `fputs()` writes the string to that stream.

Content of `output.txt`**:**

Example 2: Writing to the Console

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

```
#include <stdio.h>
int main(void)
{
    fputs("Hello students!\n", stdout);
    return 0;
}
```

What is stdout?

`stdout` is the predefined standard output stream provided by `stdio.h`.
It normally sends output to the console.

Console output:

```
Hello students!
```

Compare the Two Function Calls

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Function Call	Destination
`fputs("Hello", fp);`	File associated with `fp`
`fputs("Hello", stdout);`	Console, normally

Important

`stdout` is already available.
You do not need to call `fopen()` to use it.

Remember:

The stream determines the destination.

Quick Check!

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

What will this program display on the console?

Code

```
#include <stdio.h>
int main(void)
{
    fputs("Good", stdout);
    fputs("Morning", stdout);
    return 0;
}
```

1. Good Morning
2. GoodMorning
3. Good and Morning on separate lines
4. Nothing

Think: Does `fputs()` automatically add spaces or newlines?

Answer: Quick Check!

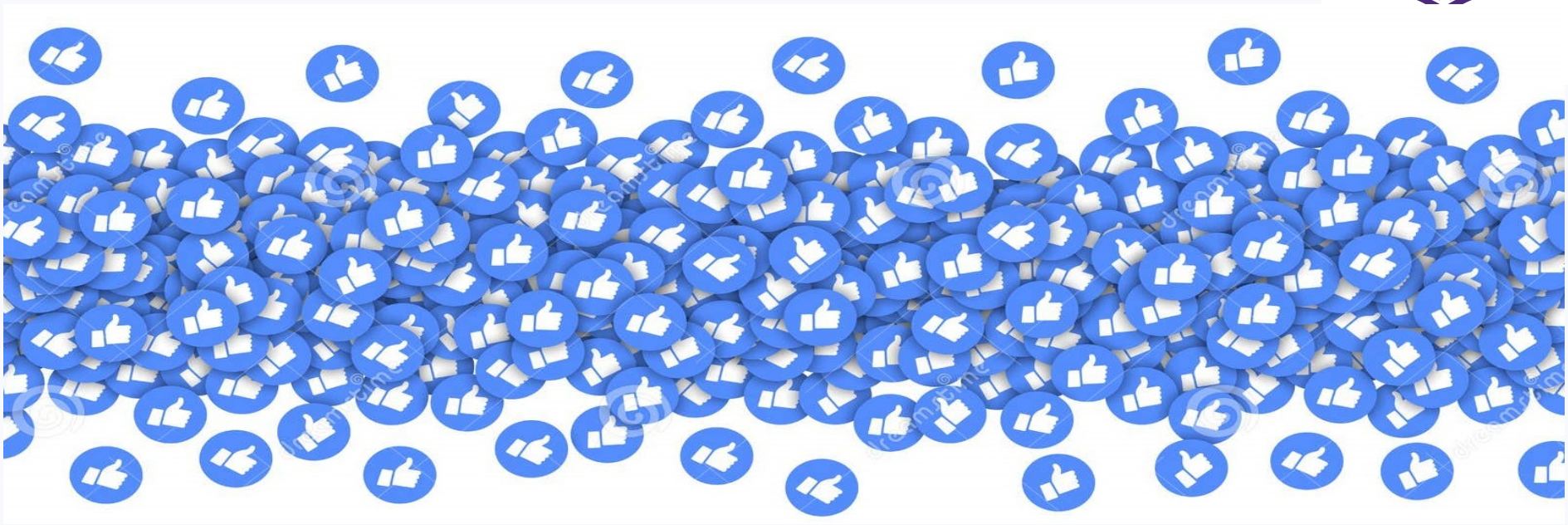
॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Option 2: GoodMorning

- First call writes `Good`.
- Second call writes `Morning` immediately after it.
- Neither call automatically adds a space or newline.

To display the words on separate lines:

```
fputs("Good\n", stdout);
fputs("Morning\n", stdout);
```



Thank you for your kind attention!!